



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии» (ИУ7)

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

Название: Обработка очередей

Дисциплина: Типы и структуры данных

Вариант: 18

Студент

ИУ7-31Б

(Группа)

Постнов С.А

(Фамилия И. О.)

Преподаватель

Силантьева А.В.

Оглавление

Условие задачи	3
Описание ТЗ.....	3
1. Описание входных данных	3
2. Описание результатов работы программы.....	3
3. Способ обращения к программе	4
4. Задача программы	4
5. Описание возможных аварийных ситуаций и ошибок пользователя....	4
Описание внутренних структур данных	5
Описание алгоритма работы	6
Сравнение эффективности двух типов очередей при разных операциях над ними	6
Выводы	7
Ответы на контрольные вопросы	7

Условие задачи

Смоделировать процесс обслуживания до выхода из ОА2 первых 1000 заявок. Выдавать на экран после обслуживания в ОА2 каждые 100 заявок информацию о текущей и средней длине каждой очереди, а в конце процесса - общее время моделирования, время простоя ОА2, количество срабатываний ОА1, среднее времени пребывания заявок в очереди. Обеспечить по требованию пользователя выдачу на экран адресов элементов очереди при удалении и добавлении элементов. Проследить, возникает ли при этом фрагментация памяти.

Описание ТЗ

1. Описание входных данных

Ограничения:

1. Любой код, не лежащий в диапазоне от 0 до 8 включительно, считается ошибочным

2. Описание результатов работы программы

В результате работы программы создаются 2 переменные типа «очередь-массив» и «очередь-список». После этого выводится меню программы, содержащее в себе следующие пункты:

- 0) Выход
- 1) Вывести очередь
- 2) Добавить элемент в очередь
- 3) Удалить элемент из очереди
- 4) Выполнить моделирование
- 5) Сравнить время выполнения операций и объем памяти при использовании двух типов очереди
- 6) Задать желаемое время обработки в первом аппарате
- 7) Задать желаемое время обработки во втором аппарате

- 8) Задать желаемый процент вероятности P (вероятность отправки обработанной заявки в конец первой очереди)

3. Способ обращения к программе

Обращение к программе происходит из командной строки путем запуска исполняемого файла «*app.exe*».

4. Задача программы

Задача данной программы заключала в себе несколько составных частей:

- Моделирование системы массового обслуживания
- Измерение времени и памяти, необходимых для добавления и удаления элемента из очереди, реализованной массивом и списком

5. Описание возможных аварийных ситуаций и ошибок пользователя

- Некорректный код меню:
 - Код возврата: 1
 - Сообщение: «Некорректный код. Попробуйте снова.»
- Ошибка выделения памяти:
 - Код возврата: 2
 - Сообщение: «Не удалось выделить память. Попробуйте снова.»
- Ошибка при пустой очереди:
 - Код возврата: 3
 - Сообщение: «Очередь пуста. Сначала добавьте элементы в нее.»
- Ошибка ввода элемента:
 - Код возврата: 4
 - Сообщение: «Не удалось получить элемент. Попробуйте снова.»
- Ошибка переполнения очереди:
 - Код возврата: 5
 - Сообщение: «Очередь полностью занята. Удалите элементы и попробуйте снова.»

Описание внутренних структур данных

Тип данных, описывающий структуру для работы с очередью в виде массива:

```
typedef struct
{
    double content[MAX_QUEUE_SIZE]; // Элементы очереди
    int len; // Длина очереди
} arr_queue_t;
```

Типы данных, описывающие структуры для работы с очередью в виде списка:

```
typedef struct node node_t;

struct node
{
    double item; // Элемент очереди
    node_t *next; // Следующий узел
};

typedef struct
{
    node_t *top; // Начало очереди
    int len; // Длина очереди
} list_queue_t;
```

Тип данных, описывающий структуру для хранения адресов освобожденных узлов очереди-списка:

```
typedef struct
{
    node_t *p_nodes[MAX_QUEUE_SIZE]; // Адреса
    int len; // Кол-во адресов
} p_node_t;
```

Тип данных, описывающий структуру для хранения информации о процессе моделирования:

```
typedef struct
{
    int first_machine_min_work_time; // Минимальное время обработки (первый аппарат)
    int first_machine_max_work_time; // Максимальное время обработки (первый аппарат)
    int second_machine_min_work_time; // Минимальное время обработки (Второй аппарат)
    int second_machine_max_work_time; // Максимальное время обработки (Второй аппарат)
}
```

```
double prob; // Вероятность
} process_info_t;
```

Тип данных, описывающий структуру для хранения результатов замерного эксперимента:

```
typedef struct measure
{
    long long unsigned time; // Время
    int mem; // Память
    int queue_len; // Длина обработанной очереди
} measurement_table;
```

Описание алгоритма работы

1. Пользователю выводится меню программы с возможностью выбрать желаемый
2. Пользователь вводит желаемый пункт меню
3. Пока пользователь не введет «0» (пункт выхода из программы), ему будет предложено вводить номера меню и выполнять желаемые действия

Сравнение эффективности двух типов очередей при разных операциях над ними

Полученная таблица замеров:

ОЧЕРЕДЬ - МАССИВ				
КОЛ-ВО ВЫПОЛНЕНИЙ	PUSH_TIME (mcs)	POP_TIME (mcs)	ПАМЯТЬ (b)	
100	1	9	8048	
250	1	54	8048	
500	2	261	8048	
1000	3	969	8048	
ОЧЕРЕДЬ - СПИСОК				
КОЛ-ВО ВЫПОЛНЕНИЙ	PUSH_TIME (mcs)	POP_TIME (mcs)	ПАМЯТЬ (b)	
100	14	1	1600	
250	89	3	4000	
500	394	6	8000	
1000	1285	11	16000	

Выводы

В ходе работы были изучены способы и алгоритмы работы с очередями, а также проведены замерные эксперименты, по результатам которых можно сделать вывод, что добавление элемента в массив-очередь происходит значительно быстрее, чем в очередь-список, так как для выполнения данной операции необходимо дойти до конца списка, что увеличивает сложность. Однако операция удаления элемента из очереди напротив выполняется дольше в очереди-массиве из-за сдвига всех элементов в цикле.

Заметим, что занимаемая очередью-массивом память больше при небольшом количестве элементов, однако при большей размерности память очереди-списка начинает значительно возрастать и превышать фиксированную память массива.

Таким образом, можно сделать вывод, что при данной реализации очередей программисту самому необходимо выбирать структуры данных, наиболее подходящие под конкретные задачи.

Ответы на контрольные вопросы

- 1) FIFO – «First In First Out»; LIFO – «Last In First Out»
- 2) Массив: память под очередь-массив выделяется сразу на максимальный размер; Список: память выделяется под каждый узел только тогда, когда он потребуется
- 3) Массив: элемент удаляется из массива, освобожден не происходит; Список: освобождается память из-под удаляемого узла, указатель на начало перемещается на новый узел
- 4) Удаление всех элементов из очереди
- 5) Все зависит от нужных при реализации условий и ограничений:
 - Особенности при реализации массивом:
 - а. Не возникает фрагментации
 - б. Размер памяти фиксирован и известен заранее

с. Возможно переполнение

- Особенности при реализации списком:
 - а. Может возникнуть фрагментация
 - б. Объем памяти ограничивается лишь оперативной
 - с. Размер памяти выделяется при необходимости

6) См. пункт 5

7) Разбиение доступной памяти на небольшие несвязные блоки называется фрагментацией памяти. Фрагментация возникает в «куче»

8) Используется для выделения памяти для списков, когда размеры блоков памяти – степени двойки

9)

- First fit: выделение памяти для первого найденного свободного участка, который имеет длину не меньше требуемой
- Best fit: выделение памяти для свободного участка памяти, который имеет точно нужную длину или минимально отличающуюся в большую сторону

10)

- Добавление в заполненную очередь
- Удаление из пустой очереди
- Освобождение памяти

11)

- Во время выделения памяти происходит резервирование места в «куче» и возвращается указатель на данный участок
- Во время освобождения памяти блок возвращается в список свободных областей